

ЛАБОРАТОРНАЯ РАБОТА: КЛИЕНТ-СЕРВЕРНЫЙ ЧАТ (FETCH API)

Цель работы: Научиться отправлять и получать данные с удаленного сервера без перезагрузки страницы. Вы создадите веб-интерфейс чата, освоите работу с асинхронным JavaScript (Promises, async/await), формами и манипуляциями с DOM.

ОТЛИЧИЕ ЭТОЙ РАБОТЫ:

Здесь не будет готового кода для копирования. Ваша задача — научиться читать документацию и искать решения. Для каждого шага указано, *что* нужно сделать и *какие запросы гуглить*. Это максимально приближено к реальной работе разработчика.

ШАГ 1: ПОДГОТОВКА ИНТЕРФЕЙСА (HTML)

Для начала нам нужен визуальный каркас нашего чата: место, где будут отображаться сообщения, и форма для отправки новых.

1. Создайте файлы `index.html`, `style.css` и `script.js`.
2. В HTML создайте пустой `<div>` с идентификатором `chat-box` (сюда мы будем скриптом добавлять сообщения).
3. Под ним создайте тег `<form>` с идентификатором `message-form`.
4. Внутри формы разместите поле ввода `<input type="text">` (дайте ему id, например `message-input`) и кнопку `<button type="submit">Отправить</button>`.
5. Напишите базовые стили в CSS, чтобы чат выглядел аккуратно (рамки, отступы, фиксированная высота для `chat-box` и `overflow-y: scroll`, чтобы сообщения можно было прокручивать).

Как проверить результат:

Откройте HTML в браузере. Вы должны увидеть пустой блок и форму под ним. При нажатии на кнопку страница должна перезагружаться (это стандартное поведение формы, позже мы его отключим).

ШАГ 2: БАЗОВАЯ НАСТРОЙКА JS И ПОДГОТОВКА АДРЕСА

Теперь нужно связать наш HTML с JavaScript и подготовить URL сервера.

1. В начале `script.js` создайте константу `baseAddress` и присвойте ей URL вашего сервера (уточните адрес у преподавателя, например: `"http://localhost:8080"` или `"https://chat.example.com"`).
2. Используя `document.getElementById` , получите ссылки на ваши HTML-элементы: форму, поле ввода и блок чата. Сохраните их в переменные.

Как проверить результат:

Напишите `console.log` для каждой переменной. Откройте консоль разработчика (F12) в браузере. Если вместо элементов вы видите `null` , проверьте, подключили ли вы скрипт *перед* закрывающим тегом `</body>` или опечатались в ID.

ШАГ 3: ПОЛУЧЕНИЕ СООБЩЕНИЙ С СЕРВЕРА (GET-ЗАПРОС)

Настало время запросить историю сообщений. Сервер отдает последние 30 сообщений по адресу `/api/v1/messages?last=30`.

1. Создайте асинхронную функцию `loadMessages()`.
2. Внутри неё используйте функцию `fetch()`, чтобы сделать GET-запрос по адресу `baseAddress + "/api/v1/messages?last=30"`.
3. Распакуйте ответ в формате JSON.
4. Выведите полученный результат в консоль (`console.log`).

Что гуглить: `JS fetch GET request async await`, `fetch response json()`.

Как проверить результат:

Вызовите функцию `loadMessages()` в самом низу вашего скрипта. Откройте страницу и посмотрите во вкладку *Console* (Консоль) или *Network* (Сеть) в DevTools. Вы должны увидеть массив объектов. *Внимательно изучите этот массив!* Как называются поля внутри объекта? `text`? `message`? `author`? Эта информация понадобится в следующем шаге.

ШАГ 4: ОТРИСОВКА СООБЩЕНИЙ В DOM

Массив данных в консоли — это хорошо, но пользователь должен видеть текст на странице.

1. Создайте функцию `renderMessages(messagesArray)`, которая принимает массив сообщений.
2. Первым делом внутри функции очистите содержимое `chat-box` (иначе старые сообщения будут дублироваться при каждом обновлении).
3. Запустите цикл по массиву `messagesArray`.
4. Для каждого сообщения создайте новый элемент (например, `<div>` или `<p>`).
5. Поместите текст сообщения внутрь созданного элемента (используйте те названия полей, которые вы увидели в консоли на предыдущем шаге).
6. Добавьте этот элемент внутрь `chat-box`.
7. Вызовите `renderMessages` внутри функции `loadMessages` после того, как получите данные от сервера.

Что гуглить: `JS очистить содержимое элемента (innerHTML)`, `JS createElement`, `JS appendChild`, `JS forEach по массиву`.

Как проверить результат:

Обновите страницу. В вашем блоке `chat-box` должен появиться список из 30 последних сообщений, отправленных другими людьми (или тестовых сообщений сервера).

ШАГ 5: ОТПРАВКА НОВОГО СООБЩЕНИЯ (POST-ЗАПРОС)

Теперь научимся писать в чат. Данные нужно отправлять на адрес

`/api/v1/messages` методом POST.

1. Повесьте обработчик события `submit` на вашу форму.
2. **ОЧЕНЬ ВАЖНО:** первым действием в обработчике отмените стандартное поведение формы, чтобы страница не перезагружалась.
3. Получите текст, который пользователь ввел в `input`. Если поле пустое — прерывайте функцию (через `return`), чтобы не отправлять пустоту.
4. Сформируйте объект данных (например: `{ text: "Мой текст" }` — сверившись со структурой, которую ожидает сервер).
5. Сделайте `fetch` запрос методом `POST`. Вам нужно будет указать заголовки (Headers) о том, что вы отправляете JSON, и превратить ваш объект данных в строку для передачи в теле (body) запроса.
6. Сразу после успешной отправки: очистите поле ввода и вызовите функцию `loadMessages()`, чтобы свежее сообщение сразу появилось на экране.

Что гуглить: `JS form submit preventDefault`, `JS fetch POST JSON body`, `JS JSON.stringify`, `JS headers Content-Type application/json`.

Как проверить результат:

Введите текст, нажмите "Отправить". Страница *не должна* перезагрузиться. В поле ввода должен пропасть текст, а в чате должно появиться ваше сообщение. Если что-то не так — открывайте вкладку *Network* в devtools браузера, ищите ваш запрос красного цвета и смотрите код ошибки сервера.

ШАГ 6 (БОНУСНЫЙ): АВТООБНОВЛЕНИЕ ЧАТА

Сейчас, чтобы увидеть новые сообщения от других людей, вам нужно обновлять страницу. Настоящие чаты работают не так.

1. Используя специальную функцию таймера, сделайте так, чтобы функция `loadMessages()` вызывалась автоматически каждые 3-5 секунд.

Что гуглить: JS выполнять функцию каждые N секунд (`setInterval`).

Как проверить результат:

Откройте ваш чат в двух разных окнах (или вкладках) браузера. Напишите сообщение в первом окне. Через пару секунд оно должно автоматически появиться во втором окне, без нажатия на кнопку "Обновить". Поздравляем, вы написали полноценный клиент-серверный чат!